

```
class Article(interface.Interface):
    title = schema.TextLine(title=u"Title")
    content = schema.TextLine(title=u"Content")
```

The model will now be:

```
class Article(grok.Model):
    interface.Implements(IArticle)
    title = ''
    content = ''
```

Let's suppose that each article can have comments. You can model the comments as a Python dictionary. However, if the number of comments is very large, you can use a container for storing comment objects.

In case the data needs to be stored in a relational database and ZODB is not an option, Grok provides an option to use an object relational mapper, like SQLAlchemy.

For completeness, it is worth noting that it is possible to use ZODB with Django!

Views

As expected, it is easy to create the add and edit forms. The code and the visual elements are well separated, thanks to Zope Page Templates (ZPT), which encourage clean separation of content, graphics design, and program code. ZPT standards are TAL (Template Attribute Language), TALES (TAL Expression Syntax) and METAL (Macro Expansion TAL). Given below is an example illustrating the display of all articles, with each row being clickable:

```
<p>Available Articles:</p>
<li tal:repeat="key python:context.
```

```
keys{*)>
    <a tal:attributes="href python:view.
url(key)" tal:content="python:key">Article
Name</a>
</li>
```

You can guess that the name of the article, which is the key for storing articles in the container, is displayed on each line. The key replaces the sample text 'Article Name' in the template. Clicking on a name would call a URL that's determined by calling a Python method.

The syntax is pure XML, and fits well into HTML. This is in contrast to some template engines, which tend to use some special characters like curly brackets, for example, to differentiate template code from HTML code (<http://docs.djangoproject.com/en/dev/topics/templates/>).

Whether using XML in a template engine makes it easier, is questionable, since the template language and HTML have to be in the same file. However, if both HTML and the template engine were replaced by something like YAML, it would definitely be a step forward to simplify coding and understanding.

To learn more about Grok, I would recommend exploring the tutorial at <http://grok.zope.org/documentation/tutorial/musical-performance-organizer-part-1/>, but only after going through the official Grok tutorial mentioned in the beginning of this article.

It is worth learning alternate frameworks like Grok even if you do not use them, simply because it can help you grok the framework that you use! 

By: Anil Seth

The author works as a consultant. Prior to consulting, Anil was a professor at Padre Conceicao College of Engineering (PCCE) in Goa, managed IT and imaging solutions for Phil Corporation Limited (Goa), supported domestic customers for Tata Burroughs/TIL, and was a researcher with IIT-K and the Indian Institute of Geomagnetism (Mumbai). You can find him online at <http://sethanil.com> and reach him via email at anil@sethanil.com.

AEM™
Enhancing Knowledge Horizon
ISO 9001:2008



redhat
CERTIFIED
TRAINING PARTNER



RHCE / RHCSS Exam Centre

**RHCE ON RHEL 6
RHCVA**

**MCTS / MCITP
Windows 2008 Server**

RHCE & RHCSS
Training By **RHCSS** Faculty
Only at **AEM** in Kolkata

ORACLE
Clustering
Shell Scripting
Solaris PHP-MySQL



STUDENTS PLACED IN

Wipro, HP, IBM, TCS, Tripod, Comtel,
Diadem, PCS, CMS, Redhat
& many more...

South Kolkata :
7/1 Jadunath Sarkar Road
Ph. : (033) 32518779, 9330925622

North Kolkata :
2/80 Dumdum Road
Ph. : 9830051236, 9830075018

www.aemk.org